

TotEm: Carbon-Aware Kubernetes Scheduling with Embodied Emissions

Nasir Asadov, Vlad C. Coroamă, Martin Roch-Neirey, Marco Zambianco, and Matthias Finkbeiner

Abstract—Data centers account for over 1% of global electricity use, yet today’s container orchestrators optimize for performance and availability while ignoring carbon emissions that vary across time and location. At the same time, manufacturing (“embodied”) emissions are a first-order contributor that existing schedulers do not model. We ask whether container scheduling can reduce total emissions—operational and embodied—without sacrificing feasibility or incurring prohibitive runtime, and how closely a practical scheduler can approach a mixed-integer linear program (MILP) benchmark. To this end, we formulate a carbon-aware placement problem that integrates operational and embodied emissions under resource and timing constraints; design TotEm (Total Emissions scheduler), a fast heuristic that minimizes a comprehensive emissions objective; and implement a global MILP as a strong benchmark, alongside a carbon-agnostic Kubernetes-like baseline. All methods are evaluated on identical workloads, horizons, carbon forecasts, and power/embodied models. Across scales, TotEm substantially reduces emissions while preserving high placement success and practical runtime. At a representative moderate-load point ($\approx 28\%$ implied average CPU utilization over 24 hours), average per-pod emissions fall from 36.6 to 24.7 g CO₂e (32.5%) relative to the carbon-agnostic baseline, while the MILP benchmark achieves 14.1 g. Runtime grows smoothly with pods and nodes and remains in single-digit seconds at our largest settings. These results establish carbon-aware container orchestration as a practical path to lower total emissions and provide a reproducible, open-source evaluation protocol for comparing heuristic and MILP approaches in sustainable cloud computing.

Index Terms—Carbon-aware computing, container orchestration, Kubernetes, sustainable computing, green cloud computing, optimization algorithms, mixed integer linear programming, global optimization.



1 INTRODUCTION

Cloud and edge data centers have spent the last decade optimizing Power Usage Effectiveness (PUE) and operational energy efficiency, achieving remarkable gains [1]. However, as operational efficiency approaches physical limits, the environmental frontier is shifting. Hardware manufacturing (“embodied” carbon) has emerged as a dominant contributor to the lifecycle footprint of distributed systems, often rivaling or exceeding operational emissions [2]. Yet, today’s container orchestration stacks, exemplified by Kubernetes [3], remain stuck in the “energy efficiency” era: they optimize primarily for resource packing and availability while ignoring both the time-varying carbon intensity of the grid and the sunk carbon cost of the hardware itself [4]. This “Embodied Gap” between what orchestrators optimize and what matters for total decarbonization motivates a principled, system-level approach that treats embodied carbon not as an external accounting detail, but as a first-class scheduling constraint. While pioneering efforts have introduced carbon-awareness into scheduling [5], [6], they predominantly focus on operational emissions, leaving the embodied gap unaddressed.

However, making container orchestration carbon-aware is nontrivial. Practical schedulers must respect placement feasibility (CPU/RAM), temporal constraints (earliest start and deadlines), and dynamic demand patterns, all while coping with carbon signals that change over time and across regions. Incorporating both operational and embodied carbon into placement decisions further complicates the objective, and any approach must remain computationally tractable at cluster scale. This raises the central research question: can we reduce total carbon emissions through orchestration decisions without sacrificing feasibility or incurring prohibitive precomputation costs?

To this end, we present TotEm, a scheduler architecture designed for Kubernetes that greedily minimizes a combined operational and embodied carbon objective under resource and timing constraints, validated here via trace-driven emulation to ensure reproducibility and explore theoretical bounds. Complementarily, we also explore a mixed-integer linear program (MILP) formulation with a lexicographic objective (first maximize successful placements, then minimize emissions) that we solve under a fixed time limit as a MILP benchmark (a carbon-emissions lower bound in our experiments); and a carbon-agnostic baseline that mirrors Kubernetes’ resource-only scoring (a carbon-emissions upper bound) for our TotEm scheduler [7]. We integrate carbon-intensity forecasts with a co-location-aware power and embodied-carbon model, enforce earliest-start and deadline constraints, and evaluate all approaches in an offline precomputation mode that simulates multi-timeslot scheduling. Our experimental design systematically varies pods (Kubernetes’ smallest schedulable unit) and nodes to assess both effectiveness and time complexity, producing

N. Asadov, V. C. Coroamă, and M. Finkbeiner are with the Department of Environmental Technology, Technische Universität Berlin, Strasse des 17. Juni 135, 10623 Berlin, Germany (e-mail: nasir.asadov@tu-berlin.de; coroama@tu-berlin.de; matthias.finkbeiner@tu-berlin.de).

M. Roch-Neirey is with the Department of Computer Sciences, School of Engineering and Architecture of Fribourg, HES-SO - University of Applied Sciences and Arts Western Switzerland (e-mail: martin@roch-neirey.com).

M. Zambianco is with Fondazione Bruno Kessler, 38123 Trento, Italy (e-mail: mzambianco@fbk.eu).

Corresponding author: Nasir Asadov (e-mail: nasir.asadov@tu-berlin.de).

Manuscript received Month Day, Year; revised Month Day, Year.

apples-to-apples comparisons across algorithms.

In a representative moderate-load setting, TotEm reduces modeled per-pod total emissions by $\approx 32\%$ versus a carbon-agnostic baseline while keeping offline precomputation in single-digit seconds (Section 5).

2 RELATED WORK

Carbon-aware workload scheduling leverages variations in the carbon intensity of electricity across *time* and *location*. Early research typically focused on *either* shifting workloads in time (temporal shifting) or across geographic locations (spatial shifting), but not both. A recent survey [8] found that most studies prior to 2023 pursued disjunct strategies—either temporal or spatial—rather than integrated approaches. Roughly 65% of works examined only one axis (time or space), though by 2024–2025 a growing share began to consider both. Beyond when and where workloads run, a complementary line of work examines how to allocate embodied (manufacturing) emissions in evaluating and optimizing schedulers. Standards such as the Software Carbon Intensity (SCI) [9] and recent studies propose time- and utilization-aware attribution; we review these approaches and their implications in Subsection 2.3. The subsequent analysis does not aim to be an exhaustive presentation of the related work. Instead, it shows — via chosen examples — how the research landscape evolved from the early, disjunct spatial or temporal scheduling towards algorithms considering the two and those considering embodied emissions as well. For the early works in particular only a few archetypical examples are shown; a thorough analysis is provided in [8].

2.1 Disjunct Temporal or Spatial Scheduling

Temporal scheduling aims to defer workloads to periods when electricity is cleaner. For example, [10] demonstrated that shifting flexible batch jobs can reduce emissions by 11–33% depending on the region and deadline slack. In practice, however, realized gains can be lower; a Kubernetes-based implementation in [11] achieved $\approx 1.3\%$ reduction for non-critical jobs using historical forecasts. These findings indicate that selectively delaying tasks can substantially shrink emissions, especially under high renewable penetration and sufficient workload flexibility.

Spatial scheduling, by contrast, exploits geographic diversity in grid carbon intensity. Recent studies demonstrate significant potential: geo-distributed service managers can reduce emissions by 25–70% depending on latency constraints [12], while location-aware VM and serverless placement strategies report gains ranging from $\approx 13\%$ to 30% [13], [14]. Early work extending Kubernetes also established the feasibility of ranking regions by real-time intensity [15]. These results confirm that spatial shifting can yield large gains when regions differ significantly in grid mix, provided available capacity at the receiving end.

2.2 Combined Spatio-Temporal Scheduling

Modern spatio-temporal schedulers seek to jointly optimize both *when* and *where* to execute workloads. A Kubernetes-based system that schedules jobs at the right cluster and

time using carbon forecasts and Quality-of-Service (QoS) constraints was proposed in [16], achieving a 33% carbon footprint reduction compared to QoS-only baselines. A dynamic multi-cloud scheduler that integrates live carbon intensity data to dispatch workloads across cloud regions and time windows was introduced in [17]. A framework that provisions and schedules workloads for carbon savings, though with less emphasis on real-time orchestration, was presented in [18].

A cautionary note is offered in [19]: many workloads show limited additional benefit from spatio-temporal scheduling beyond what single-axis strategies can achieve. Modeling across 123 regions found that simple heuristics often captured most gains, and joint shifting showed diminishing returns in carbon-stable grids or inflexible workloads.

Despite these caveats, the direction of research clearly favors holistic strategies. Kubernetes has emerged as a key enabling platform, supporting carbon-aware extensions that allow integration of forecasting, regional carbon signals, and multi-cluster placement logic [8], [16]. The fusion of container orchestration and carbon intelligence is enabling practical deployments of spatio-temporal scheduling at scale.

2.3 Embodied Emissions Allocation in Carbon-aware Scheduling

Embodied emissions (during raw material extraction, manufacturing, logistics, and end-of-life of the underlying hardware) are increasingly recognized as an important contributor to a service’s overall emissions and thus important to cover in carbon-aware scheduling. Yet, they are often omitted or uniformly amortized over calendar time. Two broad allocation strategies appear in the literature: (i) lifetime amortization (time-based), and (ii) utilization-aware attribution that splits embodied carbon by actual time-in-use and resource share.

Standards and frameworks. The SCI specification proposes attributing embodied emissions to software using the device’s total embodied emissions and two allocation factors: a time share (fraction of expected lifetime reserved/in-use) and a resource share (e.g., vCPU, memory) [9].

Recent research. Beyond uniform amortization, several works incorporate embodied carbon into scheduling and hardware-management decisions: (i) carbon depreciation models that allocate a larger share of embodied carbon to newly provisioned servers and recover idle-time impacts during active use are proposed in [20], showing that traditional accounting can inadvertently favor premature refresh; (ii) aging-aware CPU core management that extends lifetime and reduces embodied-attributed emissions in inference clusters with minimal QoS impact is demonstrated in [21]; (iii) joint embodied+operational objectives with heterogeneity and age in dispatch are formulated in [22], yielding lower total emissions under utilization-aware lifetime models; and (iv) for edge contexts, co-optimizing embodied and operational carbon under intermittent utilization and tight resource constraints is advocated by [23]. Finally, (v) EcoServe [24] addresses embodied carbon in AI inference via hardware-level strategies; while it optimizes specialized AI infrastructure, our work provides a generalized framework for the wider cloud-native ecosystem.

Implications for scheduling. (1) Pure calendar-based amortization is appropriate when only coarse information is available, but it obscures idle periods and can inflate per-workload embodied intensity. (2) With utilization traces (e.g., hourly CPU), allocating embodied emissions in proportion to time-in-use and resource share yields more faithful attribution and can shift policies toward reusing capacity, consolidating strategically, or delaying refresh. (3) Accounting for device age and heterogeneity reveals that newer hardware, despite higher performance-per-watt, carries substantial embodied carbon that must be amortized through sustained use before outperforming older devices on total emissions. (4) These effects are amplified at the edge, where intermittency and under-utilization are common.

3 CARBON-AWARE SCHEDULING STRATEGY

As stated from the outset, the aim of our carbon-aware scheduler is to minimize the carbon footprint (CF) for a random collection of nodes and pods. The CF consists of operation and embodied carbon and therefore first and foremost we describe a model that accounts for both in Section 3.1. We continue with problem formulation in Section 3.2. Since this problem is NP-hard and finding a global optimum is computationally intractable at scale, we pursue two complementary approaches: Section 3.3 introduces a MILP benchmark to approximate the theoretical lower bound, while Section 3.4 presents our TotEm scheduler designed to capture most of these benefits with practical runtime.

3.1 Carbon Accounting Models

As the CF needs to be assessed before it can be allocated, we first establish the accounting foundations: the actionable carbon-intensity signal (Section 3.1.1) is multiplied by the energy consumption of the workload. For this purpose we use a node power model that maps pod requests to energy (Section 3.1.2). We elaborate on the actual amortization and allocation of embodied emissions in (Section 3.1.3). These components compose the per-candidate emissions objective used in the problem formulation (Section 3.2).

3.1.1 Operational: Carbon Signals

We use average carbon intensity (ACI) as the actionable signal that varies by region and hour. Since standard carbon forecasts are provided at hourly resolution, we discretize the scheduling horizon into 1-hour timeslots. Forecasts are essential because our scheduler chooses not just where to run a pod, but also when, within its deadline window. With a forward view of the grid, it can push flexible jobs toward cleaner hours, steer away from expected spikes, and, when options exist, favor greener regions.

We therefore treat a time- and location-indexed forecast as a shared input to all algorithms. Both the MILP benchmark and TotEm use it to score candidate placements; TotEm also uses it to quickly rank hours and to break ties consistently across nodes. We align the forecast horizon with the scheduling horizon (12–24 h in our experiments). Forecasts are imperfect, but using the same input for all algorithms keeps comparisons fair. Section 4 details sources, coverage, and preprocessing.

TABLE 1
Hardware profiles and embodied emissions

Category	Cores	RAM (GiB)	Idle (W)	Max (W)	Embodied (kg CO ₂ e)	Life (y)
IoT	2	2	0.5	5.0	27.47	5.20
Smartphone	4	4	1.0	15.0	52.73	3.03
Laptop	8	8	10.0	150.0	231.86	4.13
Server	16	16	100.0	400.0	1230.66	3.87

3.1.2 Operational: Power Consumption of Workloads

Each node exposes a two-point power profile with an idle baseline P_{idle} and a maximum P_{max} . We model node power $P(U)$ as a piecewise linear function of aggregate CPU utilization $U \in [0, 1]$, where $P(0) = 0$ and $P(U) = P_{idle} + (P_{max} - P_{idle}) \cdot U$ for $U > 0$. This linear approximation is widely standard in cluster scheduling research [25] and sufficiently captures the dominant dynamic power component of CPU-bound workloads. This implies that the idle baseline is incurred only when at least one workload is active, and is zero otherwise.

For a pod i with CPU request c_i on a node j with total capacity C_j^{tot} , we attribute per-pod power per slot t using the same linear interpolation for the incremental component and allocate the idle baseline across concurrent workloads in proportion to CPU share:

$$p_{ijt} = (P_{max} - P_{idle}) \cdot \frac{c_i}{C_j^{tot}} + \theta_{ijt} \cdot P_{idle}, \quad (1)$$

where $\theta_{ijt} = c_i / \sum_{k \in \mathcal{A}_{jt}} c_k$ is the allocation share of pod i among the set $\mathcal{A}_{jt}$ of all pods active on node j at time t , ensuring $\sum_i \theta_{ijt} = 1$ for active slots.

3.1.3 Embodied: Emissions and Amortization

The embodied carbon calculations are based on lifecycle assessment (LCA) data from the Boavizta database [26], which aggregates manufacturer-reported environmental product declarations following ISO 14040/14044 standards.

The dataset encompasses over 200 hardware configurations from major manufacturers including Dell, Lenovo, and others, providing representative embodied carbon values and expected lifetimes across different hardware categories. We computed category-wise averages for the device types used in our experimental evaluation, as summarized in Table 1.

The core challenge in embodied carbon accounting is properly amortizing the upfront manufacturing emissions over the hardware's operational lifetime and allocating them to individual workloads. Following the Software Carbon Intensity (SCI) specification, embodied emissions attributed to a software component can be expressed as [9]:

$$M = TE \cdot TS \cdot RS,$$

where TE is total device embodied emissions from LCA, TS is the time share (e.g., reserved or in-use fraction of expected lifetime), and RS is the resource share (e.g., vCPU, memory) attributed to the software component. This structure subsumes both simple time-based amortization ($RS = 1$) and finer-grained attribution when detailed utilization or reservations are available. In our setting, TE corresponds to EC_j , TS corresponds to amortization over the

device lifetime and the pod's scheduled runtime, and RS corresponds to the per-slot allocation share θ_{ijt} . Concretely, we amortize per hour of lifetime and allocate by resource share:

$$m_{jt}^{emb} = \frac{EC_j}{L_j \times 365 \times 24} \quad (2)$$

where EC_j represents the total embodied emissions for a hardware unit (in grams CO_2e), L_j is the expected operational lifetime in years, and m_{jt}^{emb} is the amortized embodied carbon per hour of device lifetime.

For workload allocation, embodied emissions are attributed proportionally to both the time scheduled and the resource share within each active slot:

$$m_i^{emb} = \sum_{k=0}^{d_i-1} m_{j(t+k)}^{emb} \cdot \theta_{ij(t+k)} \quad (3)$$

where $\theta_{ij(t+k)}$ is the allocation share for pod i on node j at slot t (proportional to its resource share, with $\sum_i \theta_{ij(t+k)} = 1$ for active slots).

We adopt a refined, SCI-consistent allocation: we attribute embodied emissions using time-in-use and resource share derived from hourly CPU utilization. We argue that this choice is critical for *Incentive Alignment*: unlike simple time-based amortization (which charges a fixed "rent" regardless of utility), utilization-based allocation strictly aligns the scheduler's incentives with the physical reality of the hardware. By linking the carbon "cost" to actual resource consumption, the model correctly rewards maximizing the utility of the manufactured asset—thereby deferring the environmental debt of new hardware production—consistent with the economic intent of the SCI standard. We discuss the validity of this attributional approach as a proxy for consequential reductions in Section 6.

The embodied carbon calculation is integrated directly into the scheduling objective function, where it combines with operational emissions to provide a comprehensive carbon footprint assessment for each placement decision. This integrated approach ensures that both emission components influence scheduling decisions simultaneously, rather than treating them as separate optimization phases. We quantify the effect on placement patterns in Section 5.1.

3.2 Problem Formulation

We formulate the carbon-aware scheduling problem as a MILP optimization that finds optimal assignments of pods (computational tasks) to nodes (computing resources) and timeslots while minimizing total carbon emissions incorporating both operational and embodied components. The following subsections detail the problem formulation.

3.2.1 Sets, Parameters and Decision Variables

Sets. Let $\mathcal{P}$ denote the set of pods (indexed by i), $\mathcal{F}$ the set of nodes (indexed by j), and $\mathcal{T}$ the set of timeslots (indexed by t).

Parameters. We define pod i 's deadline D_i , earliest start s_i , duration d_i , slack w_i , and requests c_i, r_i . Node j has capacities C_{jt}, R_{jt} and embodied attributes. m_{ijt} represents total carbon (operational + embodied) for placing pod i on node j at time t .

Decision Variables. The binary decision variable is defined as:

$$x_{ijt} = \begin{cases} 1 & \text{if pod } i \text{ placed on node } j \text{ at time } t \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

Feasible Placements. Let $\mathcal{V}_i \subset \mathcal{F} \times \mathcal{T}$ be the set of valid node-timeslot pairs for pod i :

$$\begin{aligned} \mathcal{V}_i = \{ (j, t) \in \mathcal{F} \times \mathcal{T} \mid & t \geq s_i, t + d_i \leq D_i, \\ & C_{j(t+k)} \geq c_i \forall k \in \{0, \dots, d_i - 1\}, \\ & R_{j(t+k)} \geq r_i \forall k \in \{0, \dots, d_i - 1\} \} \end{aligned} \quad (5)$$

where the deadline timeslot $D_i = s_i + d_i + w_i$ is determined by the earliest start s_i , duration d_i , and a temporal slack window w_i allowing for scheduling flexibility.

3.2.2 Objective Function

The optimization objective minimizes total carbon emissions across all scheduled pods:

$$\text{minimize } \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} m_{ijt} \cdot x_{ijt} \quad (6)$$

where m_{ijt} represents the total carbon emissions for executing pod i on node j starting at timeslot t .

3.2.3 Carbon Emissions Model

The carbon emissions m_{ijt} incorporate both operational and embodied components over the pod's execution duration d_i :

$$m_{ijt} = \sum_{k=0}^{d_i-1} \left(m_{ij(t+k)}^{op} + m_{ij(t+k)}^{emb} \right) \quad (7)$$

where operational emissions $m_{ij(t+k)}^{op} = p_{ij(t+k)} \cdot CI_{j(t+k)}$ depend on per-slot power $p_{ij(t+k)}$ and carbon intensity $CI_{j(t+k)}$, and embodied emissions per timeslot are $m_{ij(t+k)}^{emb} = \frac{EC_j}{L_j \times 8760} \cdot \theta_{ij(t+k)}$ based on total embodied carbon EC_j , lifetime L_j (in years), and allocation share $\theta_{ij(t+k)}$ (proportional to CPU share; $\sum_i \theta_{ij(t+k)} = 1$ for active slots).

3.2.4 Constraints

Resource capacity.

$$\sum_{i \in \mathcal{P}} \sum_{k=0}^{d_i-1} c_i x_{ij, t-k} \leq C_{jt}, \quad (8)$$

$$\sum_{i \in \mathcal{P}} \sum_{k=0}^{d_i-1} r_i x_{ij, t-k} \leq R_{jt}, \quad (9)$$

for all $j \in \mathcal{F}$ and $t \in \mathcal{T}$. We take $x_{ij,t} = 0$ when $t \notin \mathcal{T}$ or $(j, t) \notin \mathcal{V}_i$.

Assignment (per pod i).

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} = 1, \quad (10)$$

with the relaxed variant given in Section 3.3.1.

Temporal feasibility is encoded by the feasible set $\mathcal{V}_i$ defined earlier (earliest start and deadline windows, plus per-slot capacity checks along the pod's duration).

3.3 Theoretical Solution: MILP Benchmark

Before Section 3.4 presents our proposed algorithm, the current section first discusses the theoretically ideal solution: the MILP that, in principle, globally optimizes scheduling decisions for minimum carbon under our model. In practice, solving this NP-hard problem to proven optimality quickly becomes intractable at scale, so we rely on a MILP implementation (Section 4.3) as a strong, but not always globally optimal, benchmark. The role of this MILP benchmark is twofold: First, to approximate lower-bound achievable emissions under identical data and constraints. Secondly, it is used in Sections 5 and 6 as one of the two reference points for our algorithm.

3.3.1 Feasibility and Relaxations

When the full MILP becomes infeasible (e.g., oversubscribed capacities or incompatible windows), we solve a relaxed variant that guarantees feasibility while preserving lexicographic priorities: (1) maximize pod placement; then (2) minimize total carbon given equal placement counts. We introduce binary slack variables y_i indicating unplaced pods and set a penalty $\lambda \gg \max m_{ijt}$ to enforce the priority ordering:

$$\text{Minimize } Z(\mathbf{x}, \mathbf{y}) = \lambda \sum_{i \in \mathcal{P}} y_i \quad (11)$$

$$+ \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} m_{ijt} \cdot x_{ijt} \quad (12)$$

subject to the softened assignment constraint

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} + y_i = 1 \quad \forall i \in \mathcal{P}. \quad (13)$$

This hierarchy ensures responsiveness in intractable instances and yields upper bounds for heuristic comparisons.

3.3.2 Computational Complexity

The MILP contains up to $|\mathcal{P}| \cdot |\mathcal{F}| \cdot |\mathcal{T}|$ binary variables over feasible pairs. The problem is NP-hard as it reduces to the NP-hard Generalized Assignment Problem (GAP) in the single-timeslot case (assigning tasks to capacity-constrained agents to minimize cost), while our formulation further extends this complexity with multidimensional resources (CPU, RAM) and temporal placement. Practical solvers (e.g., CBC or CP-SAT) employ branch-and-bound with cutting planes; performance is reasonable at moderate scales but degrades with increasing horizon and candidate sets. We therefore use solver time limits and the above relaxation to bound latency and to obtain MILP references at tractable sizes.

3.4 TotEm: Heuristic Carbon-Aware Scheduler

To address the scalability limits of the global MILP formulation, we developed TotEm, a spatio-temporal scheduling heuristic designed for practical runtimes. The principles of this algorithm were first presented in [8]; an improved—now implemented and benchmarked—version is presented in Algorithm 1 below.

The current paper introduces the further-developed version of this algorithm (Section 3.4.1). Section 4 details the

experimental protocol, while Section 5 compares TotEm against both the carbon-agnostic baseline and the MILP benchmark.

3.4.1 Algorithmic Framework

The core algorithm maintains a priority queue of pending pods ordered by deadline urgency and evaluates all feasible (node, timeslot) combinations, scoring each candidate by m_{ijt} as defined in Section 3.2.3.

Constraint handling. A candidate (j, t) is feasible only if per-slot CPU and RAM capacities hold across the pod's duration and t respects deadline windows, as defined in Section 3.2.4.

Candidate ordering. Pods are processed by smallest scheduling window (deadline minus duration), then by larger CPU, RAM, and duration (stable sort). For each pod, we prioritize timeslots with lower forecasted carbon intensity. The scheduler iterates through these pre-ordered timeslots and, for each, enumerates all candidate nodes to find the best feasible placement.

Tie-breaking and determinism. Primary score is total emissions m_{ijt} . On ties (within a tiny ϵ), we prefer tighter packing using a normalized slack metric over CPU and RAM across the pod's duration (smaller is better). We set $\epsilon = 10^{-9}$ to avoid floating-point artifacts without affecting ranking. Determinism follows from the fixed ordering of pods, timeslots, and nodes.

Search strategy. We enumerate all feasible (node, timeslot) pairs for the pod (no node sampling or early-stop); the timeslot pre-ordering biases search toward lower-carbon hours.

Implementation notes. We track per-slot residual capacity in dictionaries indexed by node and timeslot, and use stable sorts for deterministic processing. Per-iteration work remains $O(|\mathcal{F}||\mathcal{T}|)$ per pod (see Section 3.3.2).

Unschedulable case. If no feasible (j, t) exists for a pod (e.g., due to capacity or window constraints), the pod remains pending and is retried in the next scheduling round; deadlines/windows are re-evaluated each pass. No implicit relaxation or forced placement occurs in TotEm.

3.4.2 Computational Complexity

TotEm algorithm has time complexity $O(|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}|)$ for each scheduling iteration, where $|\mathcal{P}|$ is the number of pending pods, $|\mathcal{F}|$ is the number of nodes, and $|\mathcal{T}|$ is the number of timeslots. This polynomial complexity represents a significant improvement over the MILP formulation.

4 EXPERIMENTAL SETUP

To ensure clarity and reproducibility, we document the evaluation setup in the same order used to present results. Section 4.1 summarizes the system architecture (Fig. 1) to show where each scheduler runs. Section 4.2 details the testbed configuration and workload characteristics. Section 4.3 introduces the benchmarking baselines—a MILP benchmark and a carbon-agnostic scheduler. Section 4.4 outlines the evaluation methodology, including metrics, fairness protocols, and runtime measurement procedures.

Algorithm 1 TotEm Scheduler

```

1: Input: Pods  $\mathcal{P}$ , Nodes  $\mathcal{F}$ , Timeslots  $\mathcal{T}$ , Capacities, Carbon Data.
2: Init residuals  $\tilde{C}_{jt}, \tilde{R}_{jt}; S \leftarrow \emptyset$ .
3: Sort  $\mathcal{P}$  by slack  $(D_i - d_i)$ , then size/duration.
4: for  $i \in \mathcal{P}$  do
5:   Best:  $m^* \leftarrow \infty, (j^*, t^*) \leftarrow \perp, \pi^* \leftarrow \infty$ 
6:   for  $t \in [s_i, D_i - d_i]$  and  $j \in \mathcal{F}$  do
7:     if resources available for  $i$  on  $j$  at  $t \dots t + d_i - 1$ 
8:     then
9:        $\hat{m} \leftarrow \sum_{\tau=0}^{d_i-1} (m_{ij(t+\tau)}^{op} + m_{ij(t+\tau)}^{emb}); \pi \leftarrow$ 
        packing score
10:      if  $\hat{m} < m^* - \epsilon$  or  $(|\hat{m} - m^*| \leq \epsilon \text{ and } \pi < \pi^*)$ 
11:      then
12:         $m^* \leftarrow \hat{m}, (j^*, t^*) \leftarrow (j, t), \pi^* \leftarrow \pi$ 
13:      end if
14:    end if
15:  end for
16:  if  $(j^*, t^*) \neq \perp$  then  $S \leftarrow S \cup \{(i, j^*, t^*)\}$ ; update
    residuals.
17: end if
18: end for
19: return  $S$ 

```

4.1 System Architecture Overview

Our carbon-aware orchestration system is implemented as a distributed architecture that integrates with Kubernetes through a custom gRPC-based interface [27]. The system decouples carbon-aware scheduling logic from the underlying orchestration platform, enabling algorithmic experimentation.

The architecture in Figure 1 comprises four layers: (i) a client-facing ingress, (ii) a pluggable algorithm engine with three schedulers, (iii) internal data services, and (iv) external data sources. Each layer is color-coded in the figure (purple, blue, green, and orange, respectively). This separation decouples scheduling logic from data acquisition and persistence.

Architecture Components. A Go client submits requests via gRPC to the algorithm engine, which dispatches to TotEm, Oracle, or the baseline. Schedulers consume carbon forecasts (from Electricity Maps [28]), hardware metadata (from Kubernetes/Boavizta), and state storage. The system isolates algorithms from data ingestion, maintaining rolling forecasts and persistent logs for reproducibility.

Availability. We release the full implementation as open-source software [29] and provide a ‘kubect!’ plugin for cluster operators [30].

4.2 Testbed and Workloads

We evaluate all algorithms on a shared synthetic testbed with consistent infrastructure, workloads, and carbon forecasts:

Infrastructure and Nodes. Nodes are described by hardware metadata including region, embodied carbon and lifetime, and power profiles. Regions cover European grids with heterogeneous carbon intensity.

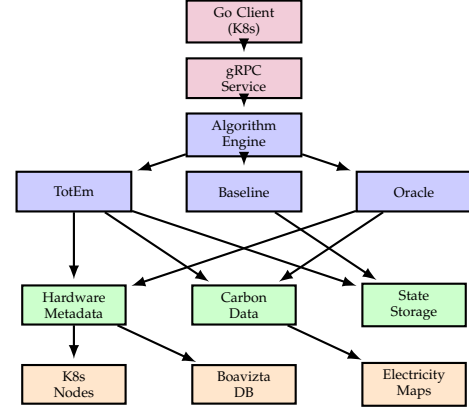


Fig. 1. Carbon-aware orchestration system architecture with component interactions and data flow. Color keys: ■ Client/Ingress ■ Core algorithms ■ Internal data services ■ External data sources

The hardware parameters in Table 1 are calibrated against representative real-world device specifications. The **IoT** profile mimics high-efficiency single-board computers similar to the Raspberry Pi Zero 2 W (typical idle $\sim 0.6W$, max $\sim 3W$) [31]. The **Smartphone** values align with mid-range mobile devices under active load (e.g., typical 1–15W power envelope) [32]. **Laptop** parameters represent high-performance workstations (e.g., Dell XPS 15 class with 130W–150W adapters) [33]. The **Server** profile models a standard dual-socket rack server (e.g., Dell PowerEdge R750 class, idling $\sim 100W$ with 300W+ peak) [34]; notably, the 16 GiB RAM configuration reflects a “compute-optimized” ratio (1:1 vCPU:RAM) similar to dedicated cloud instances (e.g., AWS c-series [35] or Azure F-series) rather than memory-abundant general-purpose servers.

Carbon Forecasts. Hourly ACI forecasts are loaded from a consolidated JSON (aligned across algorithms). Forecast horizons span 12–24 hours, enabling temporal shifting while respecting earliest/deadline constraints.

Scheduling Horizon. All methods operate over identical timeslot horizons. For fairness, feasibility checks require capacity across the entire execution duration of a pod for any candidate start slot.

Validation. A batch validator checks deadline, and capacity constraints on produced placements for all algorithms; current experiments pass with zero violations.

Workload Characteristics. Workloads are synthetically generated to simulate a heterogeneous cluster environment. Pods are assigned CPU and RAM requests sampled from a set of standard sizes (CPU: 0.1–2.0 cores; RAM: 128 MiB–2 GiB), mirroring standard cloud-native container sizes (e.g., AWS Fargate task configurations [36]). Durations are selected from $\{1, 3, 6\}$ hours, consistent with the “middle ground” of batch processing tasks (e.g., CI/CD pipelines, ML training epochs) observed in public cloud traces [37]. Deadlines are defined as $D_i = s_i + d_i + w_i$, where the slack $w_i \in \{1, 3, 6\}$ hours. This structure creates bounded scheduling windows with varying flexibility. For controlled scaling experiments, we use an “exact total” generation

strategy that distributes a fixed number of pods (ranging from 20 to 200) across the scheduling horizon to precisely control workload density. While real-world traces exist, synthetic generation allows us to systematically sweep utilization levels (from 10% to 100%) to identify the algorithm's behavior boundaries, which is difficult with fixed-density traces.

4.3 Benchmarking Baselines

The performance of the proposed algorithm is compared against two alternatives that together span the possible performance range of carbon orchestration: A plain carbon-agnostic scheduler represents the default vanilla choice. At the other end, the Oracle (MILP) scheduler, whose theoretical basis we presented in Section 3.3, approximates the best possible solution under our model.

Oracle (MILP). We instantiate the two-phase MILP benchmark as an oracle with full knowledge of all workloads arriving over the entire horizon. We use the following solver setup: OR-Tools CP-SAT by default (per-phase time limit 900 s, 8 threads), with PuLP fallbacks for HiGHS/Gurobi/CPLEX/CBC using the same time limit and a 5% relative-gap tolerance. Phase 2 uses binary activation variables to charge idle+embodied once per active (node, timeslot) with standard linking; we warm-start from Phase 1 where supported and, on timeout, run a dynamic-only fallback to return a usable bound. We log solver status, iteration counts (when available), and wall time. Because of this time limit (and, for MILP backends, the relative-gap stopping criterion), reported MILP emissions at larger scales should be interpreted as upper bounds on the true global optimum rather than certified optima.

Carbon-agnostic baseline. For a straightforward reference, we mirror Kubernetes' *filter* $\rightarrow$ *score* $\rightarrow$ *select* path: (i) filter nodes that satisfy CPU/RAM across the pod's duration at a candidate start, (ii) score feasible nodes with a LeastAllocated-like metric (equal CPU/RAM weights), and (iii) pick the best score at the earliest feasible start. It enforces earliest/deadline windows and ignores carbon in scoring. To keep behavior close to upstream, we enable node sampling and early feasible-stop so runtime and choices look like vanilla. Configuration: `percentage_of_nodes_to_score=50%`, `min_nodes_to_score=50`, `max_nodes_to_score=1000`, early stop after $k = 16$ feasible candidates. Feasibility uses a sliding-window check across the pod's duration on each node. While the baseline ignores carbon during scheduling, we calculate the resulting total emissions offline using the same shared accounting model to ensure a fair comparison. Because it prioritizes earliest-start placement with a spread-oriented (LeastAllocated-like) score, the baseline can increase fragmentation under deadline windows, reducing placement success at higher load compared to slack-aware schedulers.

4.4 Evaluation Methodology

Metrics. We report:

- **Total emissions:** operational + amortized embodied (kg CO₂e), per run and per algorithm

- **Emissions per pod:** total divided by placed pods
- **Success rate:** share of pods placed (feasibility under constraints)
- **Runtime:** median/mean per-placement execution time and end-to-end precompute time

Protocol and Fairness. To quantify carbon reductions, we evaluate three schedulers—*carbon-agnostic* (baseline), *TotEm* (carbon-aware), and *Oracle* (MILP benchmark)—over controlled workload scales (20–200 pods) and time horizons. All algorithms operate under the same constraints (earliest timeslot, CPU/Memory, node capacity/affinity) and the same emissions model (operational + amortized embodied carbon; identical regional carbon-intensity forecasts and node power profiles). We fix seeds where stochasticity is used and report the mean and standard deviation across three independent runs. Finally, we assess the robustness of our results through sensitivity analyses on forecast error and embodied carbon parameters.

Apples-to-apples comparison is non-trivial because algorithms can schedule different subsets of pods under the same load, confounding emissions with feasibility. We therefore present the coverage-inclusive view: emissions computed over all pods each algorithm successfully schedules, reflecting end-to-end impact (quality and coverage).

Runtime Measurement. We report median precompute runtime in offline mode with a 12-slot horizon. For node-scaling overlays, we fix pods at 200 and vary node count; for pod-scaling overlays, we fix nodes at 32 and vary pod count. Curves overlay multiple schedulers; error bars (when present) denote standard deviation (σ) over three replicates.

5 RESULTS

We report how embodied emissions change placement behavior (Section 5.1), quantify emissions reductions and success rates (Section 5.2 and Fig. 4), and analyze runtime scaling (Section 5.4). Finally, we assess robustness to embodied parameters and forecast errors (Sections 5.5 and 5.6).

5.1 Impact of Embodied Emissions on Scheduling

Including embodied emissions fundamentally changes placement behavior. An operational-only strategy (Fig. 2a) tends to greedily cluster workloads in timeslots or regions with the lowest grid intensity, disregarding hardware utilization efficiency. By contrast, the comprehensive objective (Fig. 2b) balances this grid-chasing with an incentive to maximize hardware utility. While operational signals might pull workloads toward a specific 'green' window, the embodied term imposes a penalty on low utilization, driving the scheduler to consolidate placements where possible—or, conversely, to utilize available capacity in slightly higher-carbon regions if doing so avoids the "embodied penalty" of spinning up new, underutilized hardware elsewhere.

Taken together, Figures 2a and 2b show that accounting for embodied emissions alters the optimization landscape. The operational-only strategy blindly chases low-CI regions, often leaving hardware underutilized. By contrast, the comprehensive objective drives consolidation to amortize the fixed embodied carbon costs, activating fewer nodes at

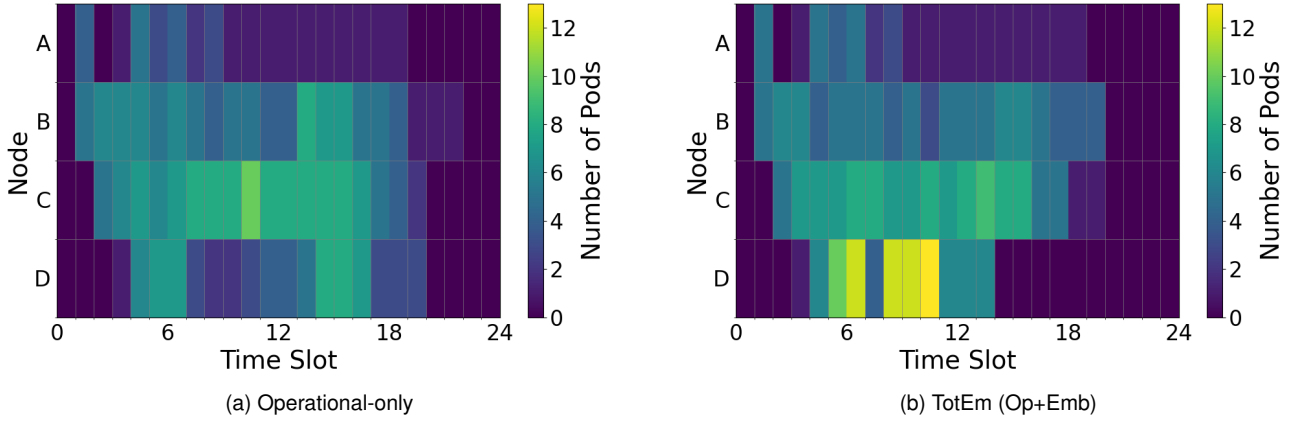


Fig. 2. Placement density comparison. (a) Operational-only scheduling concentrates work on low-carbon nodes, often underutilizing hardware. (b) TotEm drives denser packing to amortize embodied costs.

higher utilization. This trade-off can increase operational emissions slightly in some slots but reduces total (operational+embodied) emissions across the horizon; in the 100-pod runs TotEm cuts total emissions by 13.4% relative to operational-only (2.75 kg vs. 3.18 kg CO₂e).

The result aligns with our allocation model: embodied emissions are a fixed, time-amortized cost that incentivizes sustained, efficient hardware use. Ignoring them underestimates true impact and can create seemingly efficient but globally suboptimal utilization patterns.

5.2 Carbon Reduction

Figure 3a shows total emissions versus workload intensity, expressed as implied average cluster CPU utilization over a 24-hour horizon (coverage-inclusive). For each pod count, utilization is computed once from the workload generator as total requested CPU-core hours across all pods divided by the cluster’s available CPU-core hours (30 cores $\times$ 24 hours); this mapping is shared across all algorithms. Across most loads, the carbon-aware schedulers reduce emissions relative to the carbon-agnostic baseline; the Oracle benchmark consistently provides the lowest emissions, and TotEm tracks it closely in most scales. Because inclusive totals aggregate whatever each algorithm could schedule, we jointly report success rate in Figure 4 (protocol in Section 4).

Across both views we use the same emissions accounting (operational + embodied). For completeness, per-pod or per-CPU-hour normalizations and operational-only sensitivity analyses can be included in the supplement; the qualitative ordering remains stable under these variants in our experiments.

Concrete findings. At this representative utilization ($\approx 28\%$ implied average CPU utilization over 24 hours), per-pod emissions were: carbon-agnostic baseline 36.6 g CO₂e, TotEm 24.7 g ($\approx 32.5\%$), and Oracle 14.1 g ($\approx 61.5\%$). Because the MILP is solved with a 900 s per-phase time limit (and, for MILP backends, a 5% relative-gap target), the 14.1 g CO₂e value should be interpreted as the best solution found under this budget—that is, an upper bound on the true global optimum—rather than a certified optimum. This gap nevertheless illustrates that TotEm captures most of the MILP benchmark’s benefit while remaining deployable.

5.3 Placement Success Rate

Figure 4 shows scheduling success rate versus implied average cluster CPU utilization over 24 hours. At low load, all three methods are near 100%; as load increases, TotEm remains above the carbon-agnostic baseline and close to the Oracle benchmark. This gap is expected: the baseline greedily selects the earliest feasible start and favors spreading, which can fragment node–timeslot capacity and consume scarce early windows needed by tighter-deadline pods. TotEm’s slack-aware ordering and spatio-temporal search better preserve feasibility under window constraints.

5.4 Runtime Cost: Time Complexity

Figures 5a and 5b summarise runtime. It grows roughly with the number of pods and remains compatible with online use at our largest scales. The MILP scheduler is far slower and sensitive to solver limits. Following the carbon-agnostic baseline optimizations described above, the baseline series shifts down substantially in both overlays.

Setup. We report median precompute runtime in offline mode with a 12-slot horizon. Figure 5a fixes pods to 200 and varies node count; Figure 5b fixes nodes to 32 and varies pod count. Curves overlay multiple schedulers; error bars (when present) denote $\pm 1\sigma$ over three replicates.

Results. Runtime grows smoothly with both nodes and pods and remains online-friendly at our largest scales. The MILP scheduler is far slower and sensitive to solver limits, so we treat it as a benchmarking tool rather than a production scheduler. Following the carbon-agnostic baseline optimizations described above, the baseline series shifts down substantially in both overlays.

5.5 Sensitivity to Embodied Assumptions

Protocol. Embodied emissions and lifetimes in Table 1 are derived from Boavizta [26], but real deployments see variation between individual hardware configurations and management policies. To test that our conclusions do not hinge on exact values, we run a one-factor-at-a-time sensitivity study at the same representative utilization level.

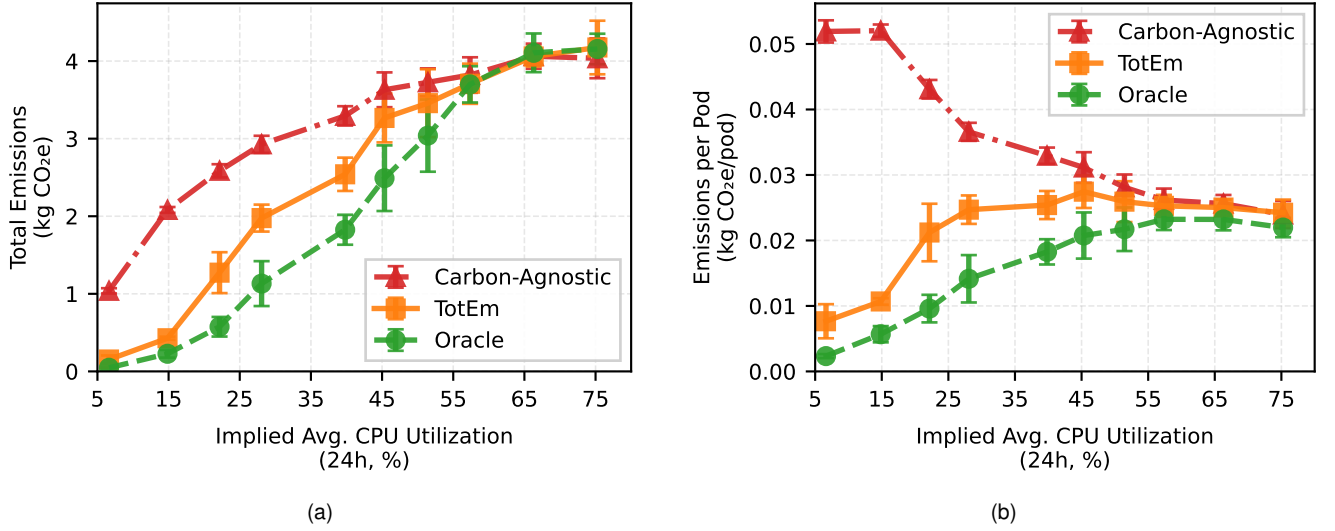


Fig. 3. (a) Total emissions vs. implied average cluster CPU utilization over 24 hours (coverage-inclusive). (b) Average emissions per placed pod (kg CO₂e/pod). Both use the shared operational+embodied model.

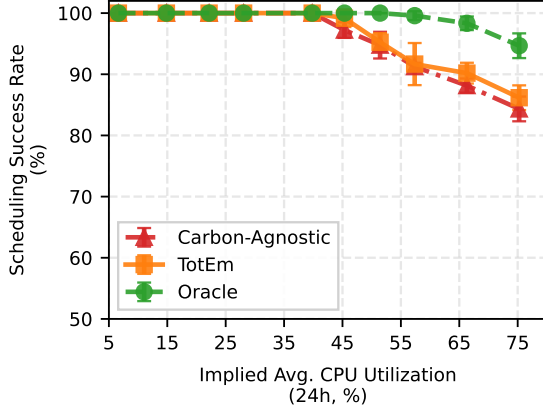


Fig. 4. Scheduling success rate vs. implied average cluster CPU utilization over 24 hours. Higher is better. Used jointly with inclusive emissions to disambiguate feasibility from carbon efficiency.

Guided by the Boavizta variability analysis, we apply uniform perturbations across all node classes: embodied emissions scaled by $\{0.60, 0.80, 1.00, 1.20, 1.60\}$ and lifetimes by $\{0.75, 1.0, 1.25\}$, averaged over independent seeds.

Findings. Figure 6 shows that these perturbations change average per-pod emissions by at most $\approx \pm 25\%$ for TotEm and $\approx \pm 17\%$ for the Oracle benchmark across embodied scaling, and within $\approx \pm 15\%$ (TotEm) / $\approx \pm 10\%$ (Oracle) across lifetime scaling—ranges consistent with the Boavizta-derived variability band. The qualitative picture remains intact: TotEm and the Oracle benchmark stay close to each other, both remain well below the carbon-agnostic baseline, and the ordering observed in Section 5.2 is preserved. This suggests that our main findings about carbon-aware scheduling and the TotEm–Oracle gap are robust to reasonable uncertainty in embodied-carbon and lifetime parameters.

5.6 Sensitivity to Forecast Error

Protocol. To test robustness we introduce controlled noise into the Electricity Maps trace and regenerate schedules using the same experimental setup as in Section 4. We inject multiplicative Gaussian noise with calibrated scale factors that reach target mean absolute percentage errors (MAPE) of 0, 5, 10, 15, and 20%; for each level we rerun the carbon-agnostic baseline and TotEm across pod counts 20, 80, 140, and 200 and three random seeds. Each scheduler observes only the noisy trace while making decisions, but we evaluate the resulting placements against the untouched trace to approximate real-world execution with forecast error. We report carbon savings relative to the optimized carbon-agnostic baseline after resampling nodes' carbon curves and recomputing operational+embodied emissions.

Findings. Figure 7 shows that TotEm exhibits remarkable robustness to forecast error across all system scales. For small experiments (e.g., 20 pods), TotEm maintains large savings even under 20% MAPE. As pod count increases, absolute savings naturally decrease due to cluster saturation, yet the savings remain stable with respect to forecast noise. This indicates that TotEm's decision-making—which ranks timeslots by carbon intensity—remains effective even when intensity values are perturbed, as the relative ordering of windows is often preserved or the best windows remain good enough. Because the carbon-agnostic baseline ignores forecasts entirely, its emissions are identical across the sweep, so all variation in the figure comes from how carbon-aware schedulers respond to imperfect predictions.

6 DISCUSSION

Our results demonstrate that incorporating both operational and embodied emissions into container orchestration is not only feasible but important for genuine sustainability. TotEm achieves substantial carbon reductions ($\approx 32.5\%$ at moderate load) relative to a carbon-agnostic baseline, tracking the Oracle benchmark effectively across most scales. In

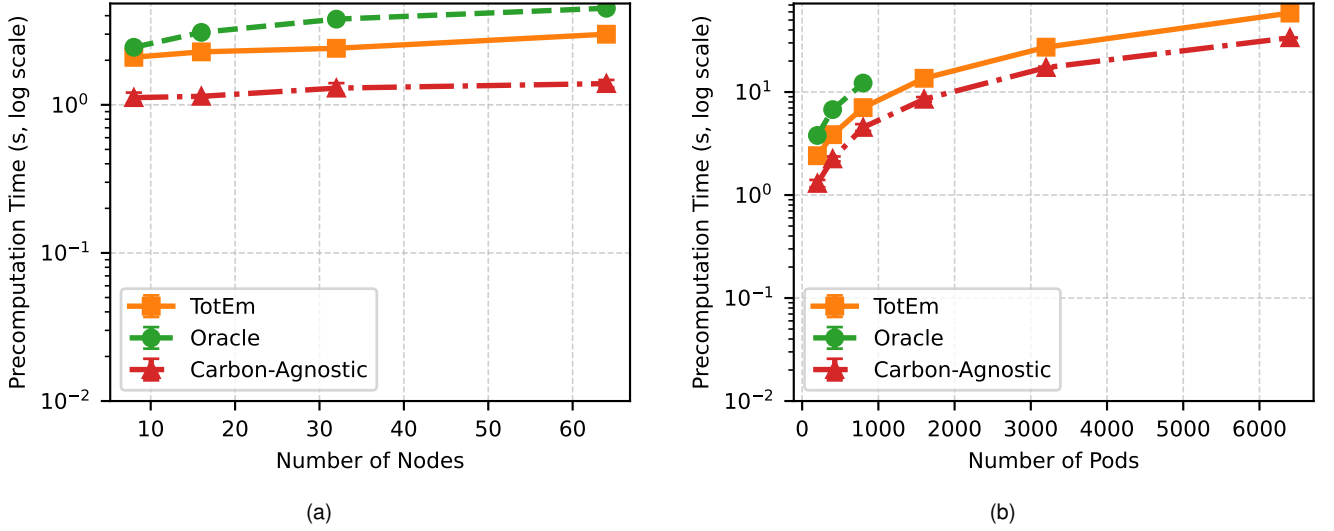


Fig. 5. Median precompute runtime vs. node count (a, 200 pods fixed) and pod count (b, 32 nodes fixed); curves overlay schedulers with $\pm 1\sigma$ error bars.

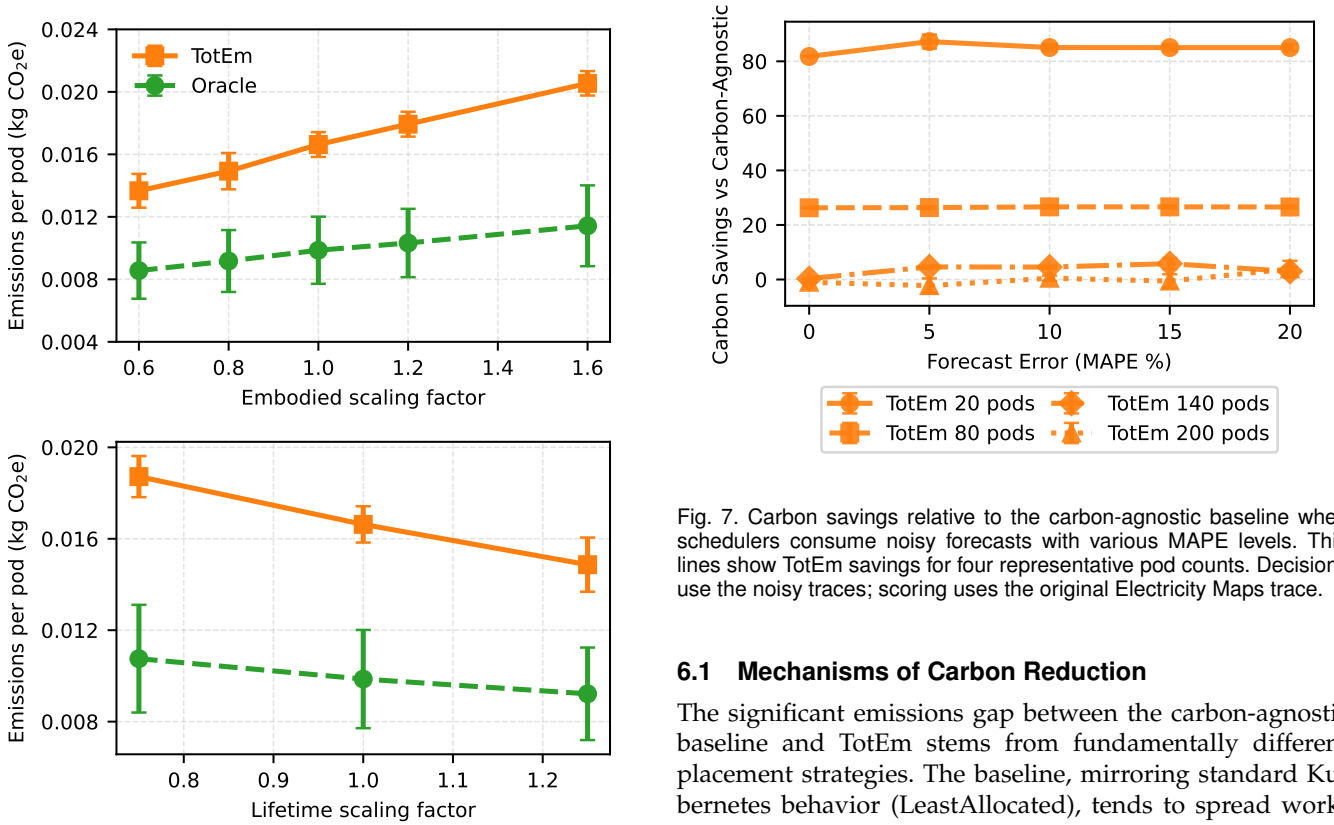


Fig. 6. Sensitivity of average per-pod emissions (kg CO₂e/pod) to embodied and lifetime scaling in the 80-pod scenario (mean with standard deviation across independent runs). Left: embodied scaling at fixed lifetime; right: lifetime scaling at fixed embodied emissions. Both TotEm and the Oracle benchmark remain within a narrow band around the baseline.

this section, we analyze the mechanisms driving these reductions, discuss the computational trade-offs, and evaluate robustness and validity.

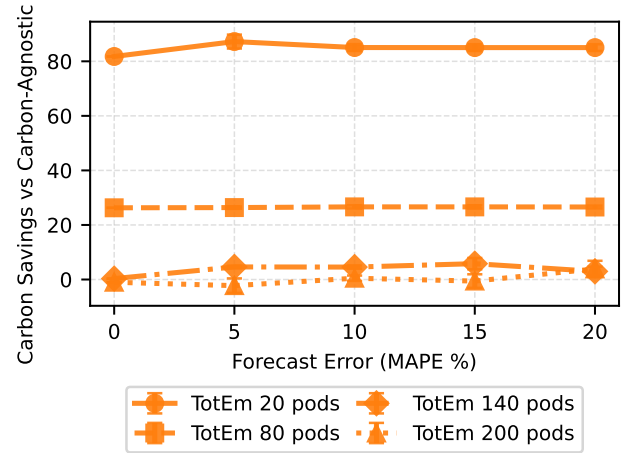


Fig. 7. Carbon savings relative to the carbon-agnostic baseline when schedulers consume noisy forecasts with various MAPE levels. Thin lines show TotEm savings for four representative pod counts. Decisions use the noisy traces; scoring uses the original Electricity Maps trace.

6.1 Mechanisms of Carbon Reduction

The significant emissions gap between the carbon-agnostic baseline and TotEm stems from fundamentally different placement strategies. The baseline, mirroring standard Kubernetes behavior (LeastAllocated), tends to spread workloads across nodes. While beneficial for failover and burst capacity, this strategy maximizes the number of active nodes, thereby incurring the idle power and embodied carbon “entry tax” of many machines simultaneously.

TotEm, by explicitly minimizing a comprehensive carbon objective, naturally favors consolidation when grid carbon intensity is high or when node-hours can be saved. As observed in Section 5.1, this shifts utilization from a “spread” pattern to a “dense” pattern, amortizing the fixed manufacturing cost of hardware over more pods per node-hour. A substantial portion of the observed carbon reduction derives from this consolidation effect incentivized by the

embodied carbon term. While traditional bin-packing aims solely to minimize fragmentation, TotEm frames this as an emissions minimization problem. By internalizing the amortized embodied carbon cost of hardware ($\approx$ manufacturing cost), TotEm naturally converges on energy-efficient consolidation patterns without requiring explicit rule-based bin-packing logic. Crucially, this is not a blind “pack everything” strategy; it is sensitive to grid intensity. The algorithm will consolidate workloads to amortize embodied carbon *except* when doing so would force execution in a dirty time window or region that outweighs the embodied savings. This dynamic balancing—a “carbon-aware bin-packing”—emerges organically from the objective function, acting as a form of “sustainable elasticity” that expands the footprint only when the grid is clean enough to justify the added hardware overhead.

6.2 Attributional vs. Consequential Impact

It is crucial to distinguish between attributional and consequential lifecycle assessment (LCA) perspectives when interpreting these results. From an *attributional* perspective, TotEm reduces the carbon allocated to each pod by packing workloads tighter, thereby lowering the computed emissions per unit of work. This is the quantity directly minimized by our objective function. From a *consequential* perspective, however, TotEm reduces global emissions only if this higher utilization allows the data center operator to provision fewer servers in the long run or to delay hardware refresh cycles. We explicitly position TotEm to optimize the attributional efficiency of the cluster as a proxy signal for these long-term consequential reductions in hardware procurement. By adhering to this proxy, the scheduler creates the necessary demand-side signal to drive supply-side decarbonization.

6.3 The Oracle Gap and Computational Practicality

While TotEm captures a large fraction of the potential savings, a gap remains between it and the MILP-based Oracle (≈ 10 g CO₂e/pod at moderate load). This gap reflects the difference between a greedy heuristic and global optimization. TotEm makes irreversible decisions per pod, potentially saturating “good” windows early, whereas the Oracle can rearrange the entire schedule to fit difficult pods into optimal slots.

However, this optimality comes at a prohibitive cost and requires future knowledge. As shown in Section 5.4, the Oracle’s runtime scales poorly, making it unsuitable for dynamic clusters. TotEm, with its polynomial time complexity and single-digit second runtimes, offers a practical trade-off: it delivers most of the sustainability benefit—changing the “order of magnitude” of emissions—while remaining deployable in real-time or near-line admission control loops.

6.4 Robustness to Real-World Uncertainty

A critical barrier to adopting carbon-aware scheduling has been the fear that forecast errors will degrade performance. Our sensitivity analysis (Section 5.6) provides reassuring evidence to the contrary. TotEm maintains consistent savings even as forecast error (MAPE) increases to 20%.

We attribute this robustness to the nature of the decision space. The scheduler relies on the *relative ranking* of time windows rather than absolute precision. Even with noise, a “clean” night-time window generally remains cleaner than a “dirty” evening peak. As long as the signal preserves the coarse-grained structure of the grid’s daily cycle, TotEm can successfully shift work to lower-carbon periods. Similarly, Section 5.5 shows that while exact embodied carbon values shift the absolute numbers, the relative ordering of algorithms and the benefits of consolidation remain stable, insulating the approach from supply-chain data uncertainties.

6.5 Threats to Validity

Internal Validity. Our experiments rely on synthetic workloads. While we modeled requests and durations based on common cluster traces, real production environments may exhibit burstier arrival patterns or complex dependencies (e.g., microservice chains) that we did not model. We mitigate this by sweeping a wide range of load factors to ensure the trends hold across utilization regimes.

External Validity. We utilize ACI as our optimization signal. Some economists argue that Marginal Emissions Factors (MEF) more accurately reflect the immediate grid impact of load changes. While TotEm is agnostic to the signal source—it minimizes whatever time-series it is given—the magnitude of savings might differ under MEF. Furthermore, our offline precomputation assumes perfect knowledge of job arrival (batch processing); in an online setting, the “greedy” penalty might increase as the scheduler cannot see future arrivals, though this is a standard limitation of online algorithms.

Construct Validity. Our model assumes that “embodied carbon” can be linearly amortized over time and utilization. While consistent with the SCI standard, this is an accounting convention. It does not physically “emit” carbon during the job’s run but rather attributes a share of the sunk cost. We argue this is the correct incentive for software design: it encourages maximizing the utility of manufactured devices, thereby deferring the need for new hardware production. Finally, our current model focuses on server-side operational and embodied emissions; it does not explicitly model network energy consumption (data transfer) or the embodied carbon of networking equipment (switches, routers), which may be significant for data-intensive workloads.

7 CONCLUSION AND FUTURE WORK

This paper showed that container orchestration can reduce *total* emissions—operational and embodied—without sacrificing feasibility or incurring prohibitive runtime. Across controlled scales, TotEm consistently lowers emissions relative to a carbon-agnostic baseline while closely tracking the Oracle benchmark. At a representative moderate-load point ($\approx 28\%$ implied average CPU utilization over 24 hours), average per-pod emissions dropped from 36.6 to 24.7 g CO₂e (-32.5%) versus baseline, with the Oracle reaching 14.1 g. Runtime grows smoothly with pods and nodes and remains within single-digit seconds at our largest settings, indicating practicality for precompute or nearline use. Furthermore,

our sensitivity analysis demonstrates that TotEm's carbon savings are robust to forecast errors up to 20% MAPE, validating the reliability of carbon-aware scheduling even with imperfect grid intelligence.

For practitioners, these results suggest an actionable path: integrate forecasted carbon signals and amortized embodied costs into placement, and deploy a fast heuristic to capture most of the achievable benefit. By optimizing this attributional efficiency, operators can generate the demand-side signals necessary to drive long-term consequential reductions in hardware procurement. Our evaluation protocol—shared signals, models, constraints, and coverage-inclusive reporting—provides a reproducible basis for comparing algorithms and for tuning trade-offs between feasibility, carbon efficiency, and latency.

Future work will: (i) integrate additional environmental indicators (in particular water footprint); and (ii) evaluate on production traces and in live clusters with online admission. Together, these steps should help moving carbon-aware orchestration from controlled studies toward robust production deployments.

ACKNOWLEDGMENTS

The authors would like to extend their gratitude to Electricity Maps for providing access to their carbon intensity data, which was invaluable for the development of this work. We further thank Philipp Wiesner, Silvio Cretti, and Sébastien Rumley for fruitful discussions early in the development process.

REFERENCES

- [1] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, 2020.
- [2] Z. Cao, X. Zhou, X. Wu, Z. Zhu, T. Liu, J. Neng, and Y. Wen, "Data center sustainability: Revisits and outlooks," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 236–248, 2024.
- [3] K. Penfound and C. Nickerson, "Production-grade container orchestration," [Online]. Available: <https://kubernetes.io/>, 2024.
- [4] Ember, "Global electricity review 2024," [Online]. Available: <https://ember-climate.org/insights/research/global-electricity-review-2024/>, 2024.
- [5] A. Radovanovic, J. Dean *et al.*, "Carbon-intelligent computing at google," [Online]. Available: <https://ai.googleblog.com/2020/05/carbon-intelligent-computing-at-google.html>, 2020.
- [6] L. Gu, W. Zhang, Z. Wang, D. Zeng, and H. Jin, "Service management and energy scheduling toward low-carbon edge computing," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 1, pp. 109–119, 2023.
- [7] Kubernetes Authors, "Scheduling framework," [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>, 2025.
- [8] N. Asadov, V. C. Coroamă, M. Franzil, S. Galantino, and M. Finkbeiner, "Carbon-aware spatio-temporal workload shifting in edge-cloud environments: A review and novel algorithm," *Sustainability*, vol. 17, no. 14, 2025.
- [9] Green Software Foundation, "Software carbon intensity (sci) specification," [Online]. Available: <https://sci.greensoftware.foundation/>, 2024.
- [10] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud," in *Proc. Middleware*, 2021.
- [11] T. Piontek, K. Haghsheenas, and M. Aiello, "Carbon emission-aware job scheduling for kubernetes deployments," *J. Supercomput.*, 2023.
- [12] A. Souza, S. Jatoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh, A. Ali-Eldin, D. Irwin, and P. Shenoy, "CASPER: Carbon-aware scheduling and provisioning for distributed web services," in *Proc. IGSC*, 2024, pp. 67–73.
- [13] M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, and O. Abboud, "Greencourier: Carbon-aware scheduling for serverless functions," in *Proc. WoSC*, 2023, pp. 18–23.
- [14] S. Rawas, A. Zekri, and A. El-Zaart, "LECC: Location, energy, carbon and cost-aware vm placement model in geo-distributed dcs," *Sustain. Comput. Inform. Syst.*, vol. 33, p. 100649, 2022.
- [15] A. James and D. Schien, "A low carbon kubernetes scheduler," in *Proc. ICT4S*, ser. CEUR Workshop Proc., vol. 2382, 2019.
- [16] T. Bahreini, A. N. Tantawi, and O. Tardieu, "Caspian: A carbon-aware workload scheduler in multi-cluster kubernetes environments," in *Proc. MASCOTS*, 2024.
- [17] B. M. Beena, P. C. Ranga, T. S. S. Manideep, S. Saragadam, and G. Karthik, "A green cloud-based framework for energy-efficient task scheduling using carbon intensity data for heterogeneous cloud servers," *IEEE Access*, vol. 13, pp. 73 916–73 938, 2025.
- [18] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, "Carbonflex: Enabling carbon-aware provisioning and scheduling for cloud clusters," 2025, arXiv:2505.18357.
- [19] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, "On the limitations of carbon-aware temporal and spatial workload shifting in the cloud," in *Proc. EuroSys*, 2024, pp. 924–941.
- [20] S. Ji, Z. Yang, A. K. Jones, and P. Zhou, "Towards datacenter environmental sustainability using carbon depreciation models," 2025, arXiv:2403.04976.
- [21] T. B. Hewage, S. Ilager, M. R. Read, and R. Buyya, "Aging-aware cpu core management for embodied carbon amortization in cloud llm inference," 2025, arXiv:2501.15829.
- [22] Y. Zhang, Y. Song, and S. Sahoo, "Carbon and reliability-aware computing for heterogeneous data centers," 2025, arXiv:2504.00518.
- [23] A. M. Panteleaki, V. Paramanayakam, V. Pentsos, A. Karatzas, S. Tragoudas, and I. Anagnostopoulos, "A vertical approach to designing and managing sustainable heterogeneous edge data centers," 2025, arXiv:2506.01712.
- [24] Y. Li, Z. Hu, E. Choukse, R. Fonseca, G. E. Suh, and U. Gupta, "EcoServe: Designing carbon-aware ai inference systems," 2025, arXiv:2502.05043.
- [25] X. Fan, W.-D. Weber, and L. A. Barroso, "Power provisioning for a warehouse-sized computer," in *ACM SIGARCH Comput. Archit. News*, vol. 35, no. 2, 2007, pp. 13–23.
- [26] Boavizta, "Boavizta/environmental-footprint-data," [Online]. Available: <https://github.com/Boavizta/environmental-footprint-data>, 2023.
- [27] S. Cretti, "fogatlas-k8s / algorithms," [Online]. Available: <https://gitlab.fbk.eu/fogatlas-k8s/algorithms>, 2024.
- [28] Electricity Maps, "Live 24/7 co2 emissions of electricity consumption," [Online]. Available: <http://electricitymap.tmrow.co>, 2024.
- [29] N. Asadov, "nasadov/carbon-aware-orchestrator," [Online]. Available: <https://github.com/nasadov/carbon-aware-orchestrator>, 2025.
- [30] S. Cretti and N. Asadov, "nasadov/kubectl-carbon," [Online]. Available: <https://github.com/nasadov/kubectl-carbon>, 2025.
- [31] Raspberry Pi Ltd, *Raspberry Pi Zero 2 W Product Brief*, 2021. [Online]. Available: <https://datasheets.raspberrypi.com/rpizero2/raspberry-pi-zero-2-w-product-brief.pdf>
- [32] Samsung Electronics, *Samsung Galaxy S24 Series Technical Specifications*, 2024. [Online]. Available: <https://news.samsung.com/global/enter-the-new-era-of-mobile-ai-with-samsung-galaxy-s24-series>
- [33] Dell Technologies, *Dell XPS 15 9510 Setup and Specifications*, 2021. [Online]. Available: <https://dl.dell.com/content/manual46844474-xps-15-9510-setup-and-specifications.pdf>
- [34] —, *Dell EMC PowerEdge R750 Technical Guide*, 2021. [Online]. Available: https://i.dell.com/sites/csdocuments/Product_Docs/en/poweredge-r750-technical-guide.pdf
- [35] Amazon Web Services, "Amazon ec2 c7g instances," [Online]. Available: <https://aws.amazon.com/ec2/instance-types/c7g/>, 2024.
- [36] —, *AWS Fargate Pricing*, 2024. [Online]. Available: <https://aws.amazon.com/fargate/pricing/>
- [37] M. Shahradd, R. Fonseca, I. n. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX ATC*, 2020, pp. 205–218.